Computer Science & IT

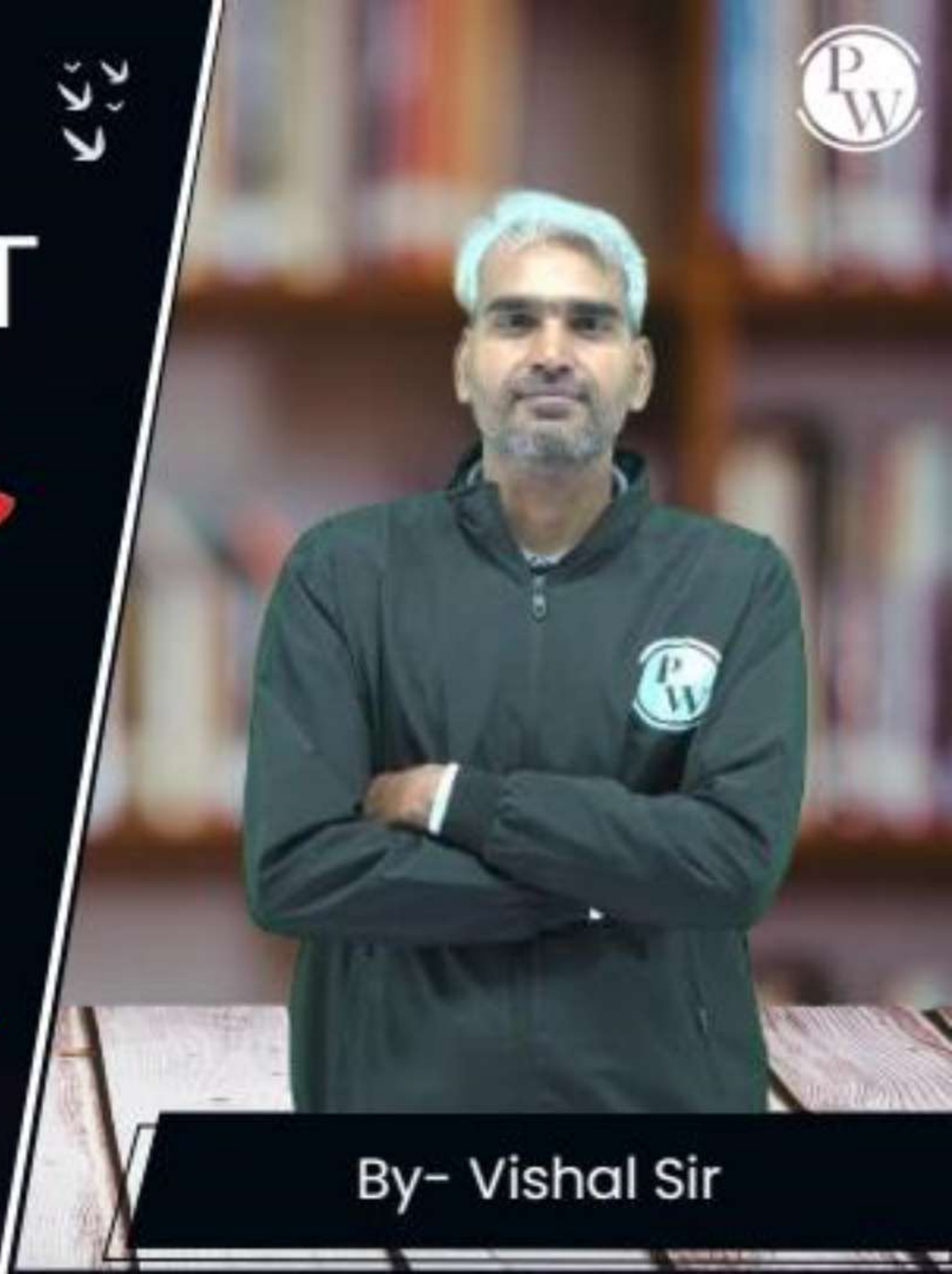


Database Management System

File organization And Indexing

+ Last lecture of Transactions & Concurrency Control.

Lecture No. 01



By- Vishal Sir

Recap of Previous Lecture



- **Topic** Basic time stamp ordering protocol
- **Topic** Time stamp ordering protocol with Thomas write rule
- **Topic** Wait-die protocol for deadlock prevention
- **Topic** Wound-wait protocol for deadlock prevention

Slide

Topics to be Covered



- Topic: Time stamp ordering protocol with Thomas write rule
- Topic: Wait-die protocol for deadlock prevention
- Topic: Wound-wait protocol for deadlock prevention
- Topic: Database, File and Records
- Topic: Organization of Records

Slide



Topic : Time stamp ordering protocol with Thomas write rule

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

Case ① When transaction T_1 issue a Read(A) oph

(i)

T_1	T_2
	R(A)
R(A)	

(ii)

T_1	T_2
	W(A)
R(A)	

These three case will be exactly same as B.T.S.O.P.

Case ② When transaction T_1 issue a Write(A) oph

(i)

T_1	T_2
	R(A)
W(A)	

(ii)

T_1	T_2
	W(A)
W(A)	

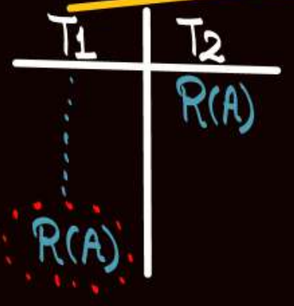
This case will be different from B.T.S.O.P.

Time stamp ordering protocol with Thomas write Rule :-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

① When transaction T_1 issue a Read(A) opⁿ

(i)

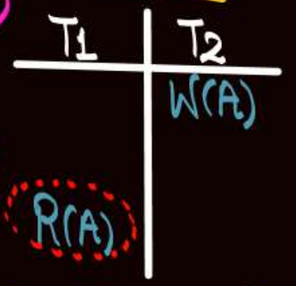


$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

If T_1 is allowed to perform this R(A) opⁿ then also the behaviour of schedule will be view equivalent to serial schedule as per time stamp ordering of transactions

∴ T_1 is allowed to perform R(A) opⁿ

(ii)



$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

If T_1 is allowed to perform this R(A) opⁿ then the behaviour of schedule will not be view equivalent to serial schedule as per time stamp ordering of transactions

∴ T_1 is not allowed to perform R(A) opⁿ

i.e. Rollback T_1

* If T_1 issue a R(A) opⁿ and $WTS(A) > TS(T_1)$, then Rollback T_1

Time stamp ordering protocol with Thomas write Rule :-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

② When transaction T_1 issue a Write(A) opn

(i)

T_1	T_2
	R(A)
W(A)	

Rollback

$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

If transaction T_1 is allowed to perform this W(A) opn then behaviour of schedule will not be view equivalent to serial schedule as per time stamp ordering of transactions

∴ Transaction T_1 is not allowed to perform this W(A) opn, and we rollback T_1

If T_1 issue a W(A) opn and $RTS(A) > TS(T_1)$ then Rollback T_1

(ii)

T_1	T_2
	W(A)
W(A)	

$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

Time stamp ordering protocol with Thomas write Rule :-

Let T_1 & T_2 are two transactions such that $TS(T_1) < TS(T_2)$

② When transaction T_1 issue a Write(A) oph

(i)

T_1	T_2
	R(A)
W(A)	

Rollback

$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

If transaction T_1 is allowed to perform this W(A) oph then behaviour of schedule will not be view equivalent to serial schedule as per time stamp ordering of transactions

∴ Transaction T_1 is not allowed to perform this W(A) oph, and we rollback T_1

If T_1 issue a W(A) oph and $RTS(A) > TS(T_1)$ then Rollback T_1

(ii)

T_1	T_2
	W(A)
W(A)	

$TS(T_1) = 10$
$TS(T_2) = 20$
∴ Time stamp ordering is $T_1 \rightarrow T_2$

If T_1 is allowed to perform this W(A) oph, then behaviour of schedule will not be view equivalent to serial schedule as per time stamp ordering of transactions

But, if we skip (ignore) this W(A) of transaction T_1, then schedule produced will be "view equivalent" to serial schedule as per time stamp ordering of transactions.

* If transaction T_1 issue a W(A) oph and $WTS(A) > TS(T_1)$, then ignore (skip) this W(A) oph by transaction T_1 and continue with other operations.

Serial schedule as per time stamp ordering (i.e. $T_1 \rightarrow T_2$)

T_1	T_2
W(A)	
	W(A)

⟹ if we ignore W(A) oph by transaction T_1 then it becomes

T_1	T_2
	W(A)

Time stamp ordering Protocol with Thomas Write Rule :-

Let T_1 & T_2 are two transactions s.t. $TS(T_1) < TS(T_2)$
i.e, T_1 is older than T_2

① Let T_1 issue Read(A) opn :-

- ⓐ If $WTS(A) > TS(T_1)$, then Rollback T_1
- ⓑ Otherwise, T_1 is allowed to perform the Read(A) opn
  ∴ T_1 will perform Read(A) opn.
  And set $RTS(A) = Max(RTS(A), TS(T_1))$

② Let T_1 issue Write(A) opn :-

- ⓐ If $RTS(A) > TS(T_1)$, then Rollback T_1.

and

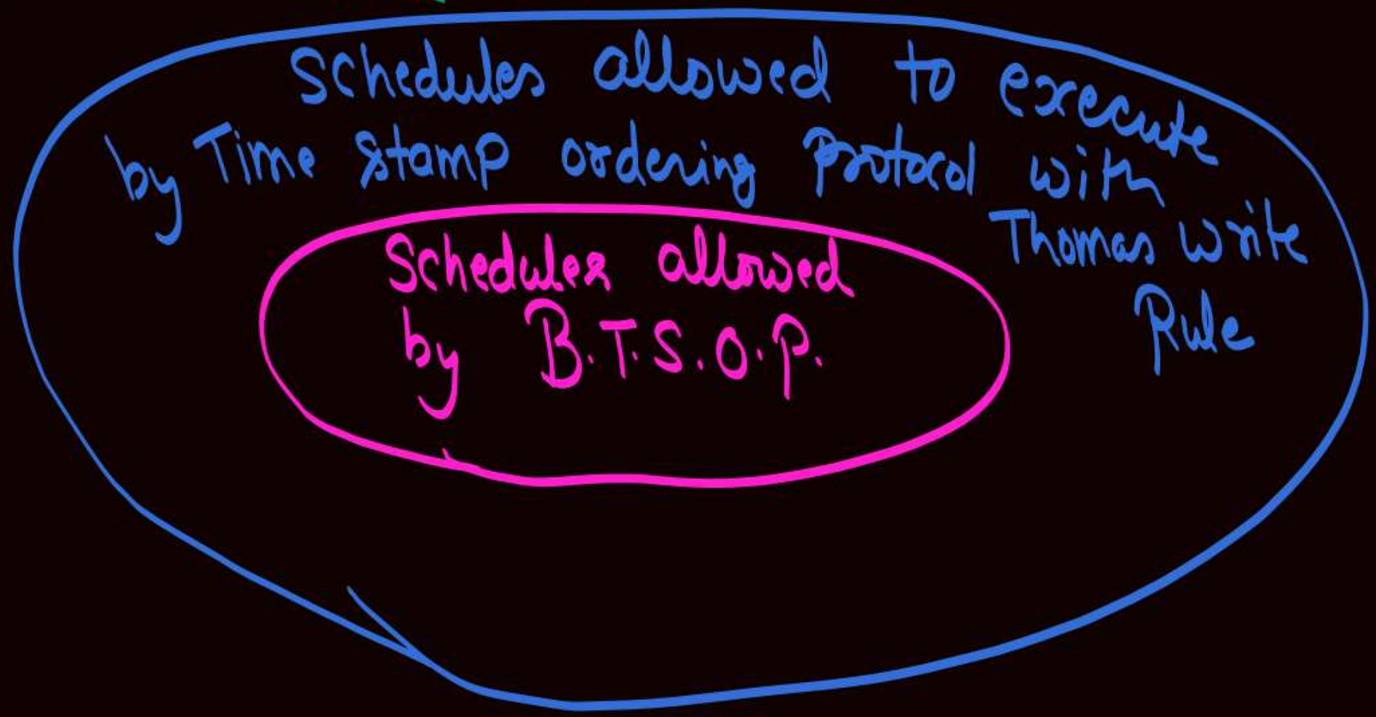
- ⓑ If $WTS(A) > TS(T_1)$, then ignore this w(A) opn of transaction T_1
  And Continue with the other operations of transaction T_1

and

- ⓒ Otherwise, T_1 is allowed to perform Write(A) opn.
  ∴ T_1 will perform the write(A) opn
  And Set $WTS(A) = TS(T_1)$

Notes :-

① Every schedule which is allowed by B.T.S.O.P. is allowed to execute using Time stamp ordering Protocol with Thomas write Rule, But Converse of the statement need not be true



Note:- (2)

If A schedule is allowed to execute using Time Stamp ordering protocol with Thomas write Rule, then schedule produced by Time Stamp ordering protocol with Thomas write Rule will be view equivalent to the serial schedule based on time stamp ordering of the transactions.

Note :- ③ Both B.T.S.O.P & Time stamp ordering protocol with Thomas write rule are free from deadlock, but starvation is possible

④ Schedules allowed by B.T.S.O.P and/or Time stamp ordering protocol with Thomas write rule may suffer from irrecoverability, cascading rollback and lost-update problem, To solve that problem we can implement Strict time stamp ordering protocols.



Topic : Deadlock prevention algorithms

* We can use the Concept of time stamps assigned by DBMS to prevent the deadlock in lock based Concurrency Control Protocols.

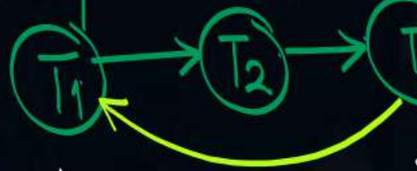
If dependency graph is cyclic, then deadlock may be possible

Dependency graph



represent that T_1 requires the resource held by T_2

If dependency graph is of the type



then it is Cyclic dependency graph and Hence deadlock may be possible

If we can prevent from the formation of cycles in the dependency graph, then we can prevent from deadlock



Topic : Deadlock prevention algorithms

* There are two algorithms (Protocols) that can be used to prevent the the formation of cycles in the dependency graphs, and hence can be used to prevent the deadlock in lock based protocols.

Those algorithm are

1. Wait - die protocol
2. Wound - wait protocol.



Topic : Deadlock prevention algorithms

Old — Young

① Wait - Die Protocol
(Suicide i.e. Rollback)

② Wound - Wait Protocol
(Kill/Murder i.e. Kill the other one)

Killing the younger one

Names are wrt Old — young Pair

We will always rollback the "younger" transaction



Topic : Wait-die algorithms

Old – young

Old one will wait

Younger one will die

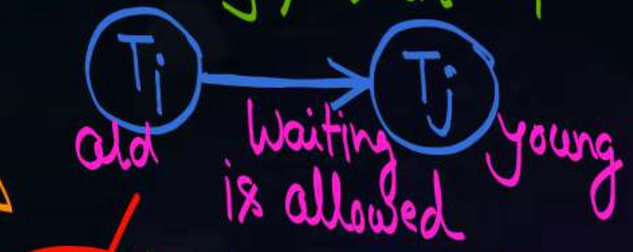
If the old transaction requires the resource held by younger transaction, then old transaction is allowed to wait

If the younger transaction requires the resource held by old transaction, then younger one will Rollback (i.e. die)

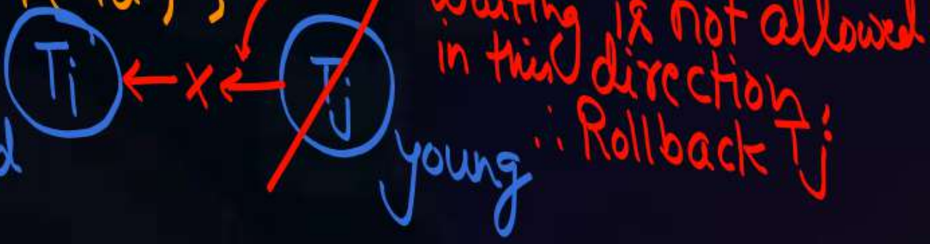
Let T_i & T_j are two transactions such that $TS(T_i) < TS(T_j)$

i.e. T_i is old & T_j is young

1. If transaction T_i (old transaction) requires the resource held by T_j (young transaction), then T_i is allowed to wait for T_j



2. If transaction T_j (young) requires the resource held by T_i (old), then rollback T_j





Topic : Wound-wait algorithms

old young

Old one will wound the young one

If old transaction requires the resource held by young one, then they will wound the younger transaction

young one will wait for old one

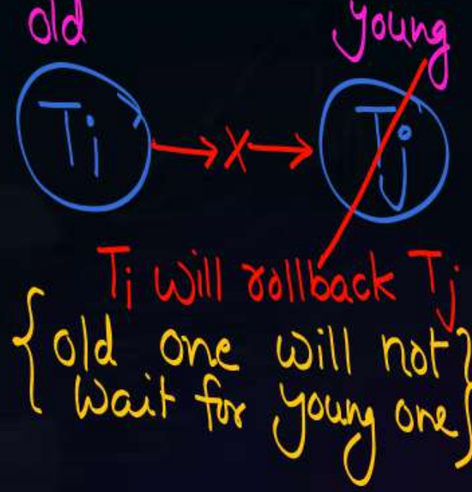
→ If young transaction requires the resource held by old transaction, then young transactions are allowed to wait

---

Let T_i & T_j are two transactions such that $TS(T_i) < TS(T_j)$

i.e. T_i is old & T_j is young

① If transaction T_i (old) requires the resource held by T_j (young), then T_i will rollback T_j



② If transaction T_j (young) requires the resource held by T_i (old) then T_j is allowed to wait



T.S. : 10 20 30 40

T_1 T_2 T_3 T_4

wait-die



not allowed (Rollback T_4)

Note:-

- Using Wait-die & Wound-wait we can Prevent from deadlock in lock based protocols, but Starvation will still be possible.

File organization & Indexing

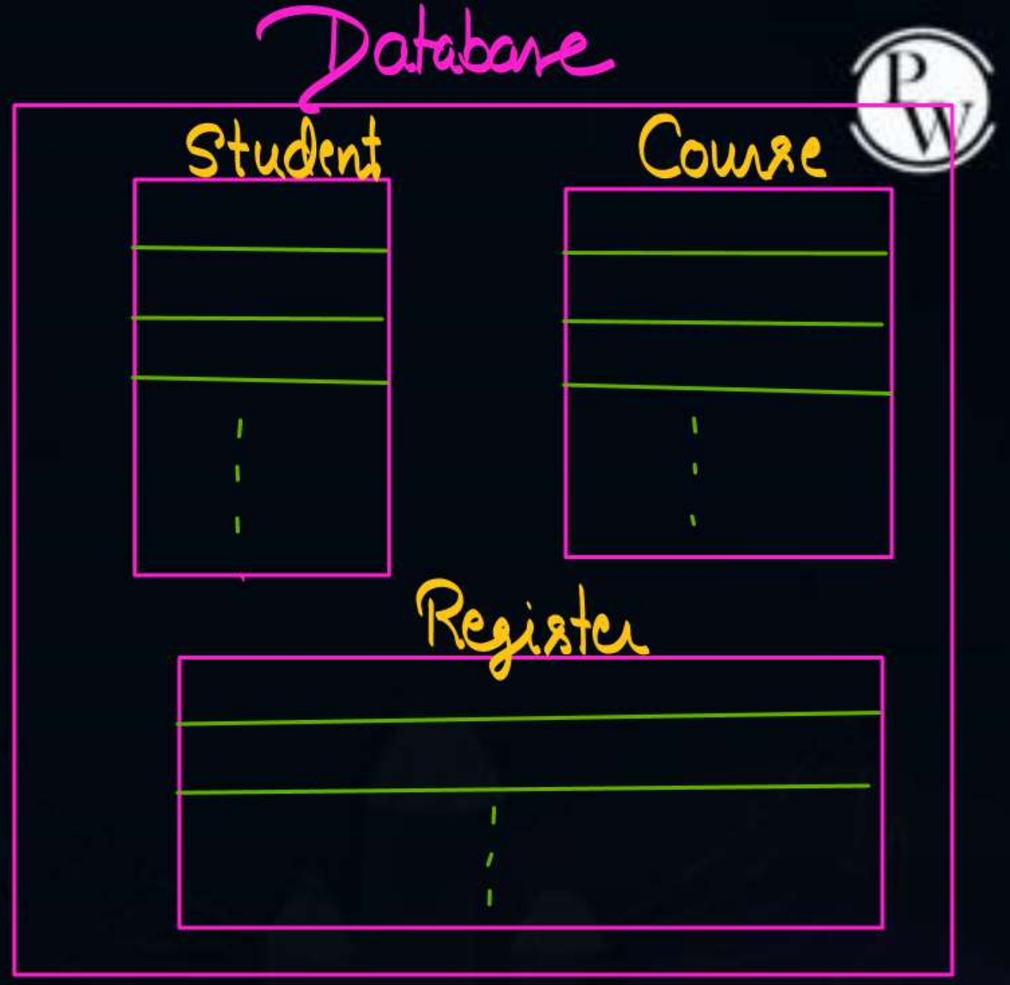


Topic : Database-File-Records

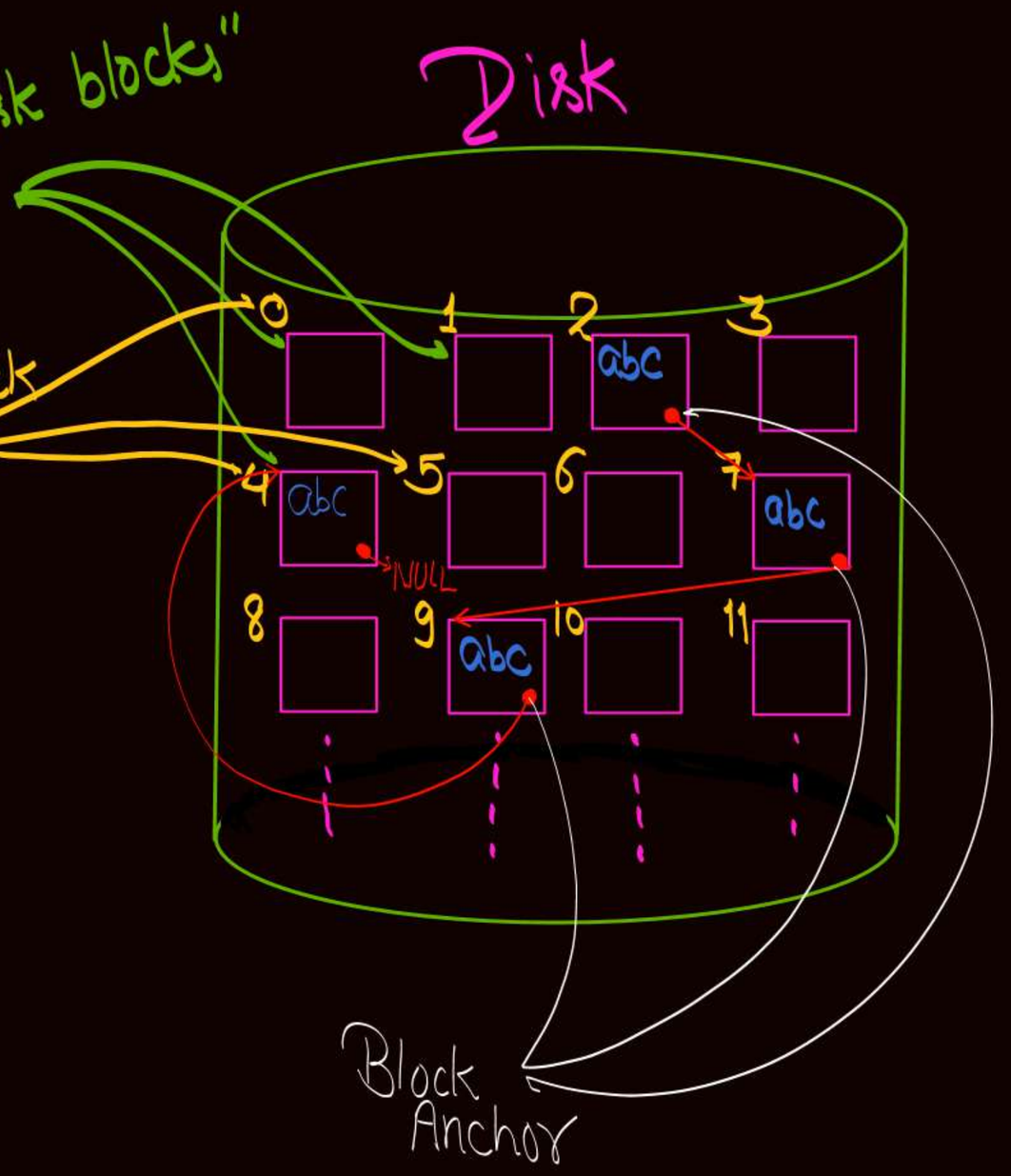
* Database is a collection of files
* Files are used to store the records.
* Disk blocks are used to store the records of the file

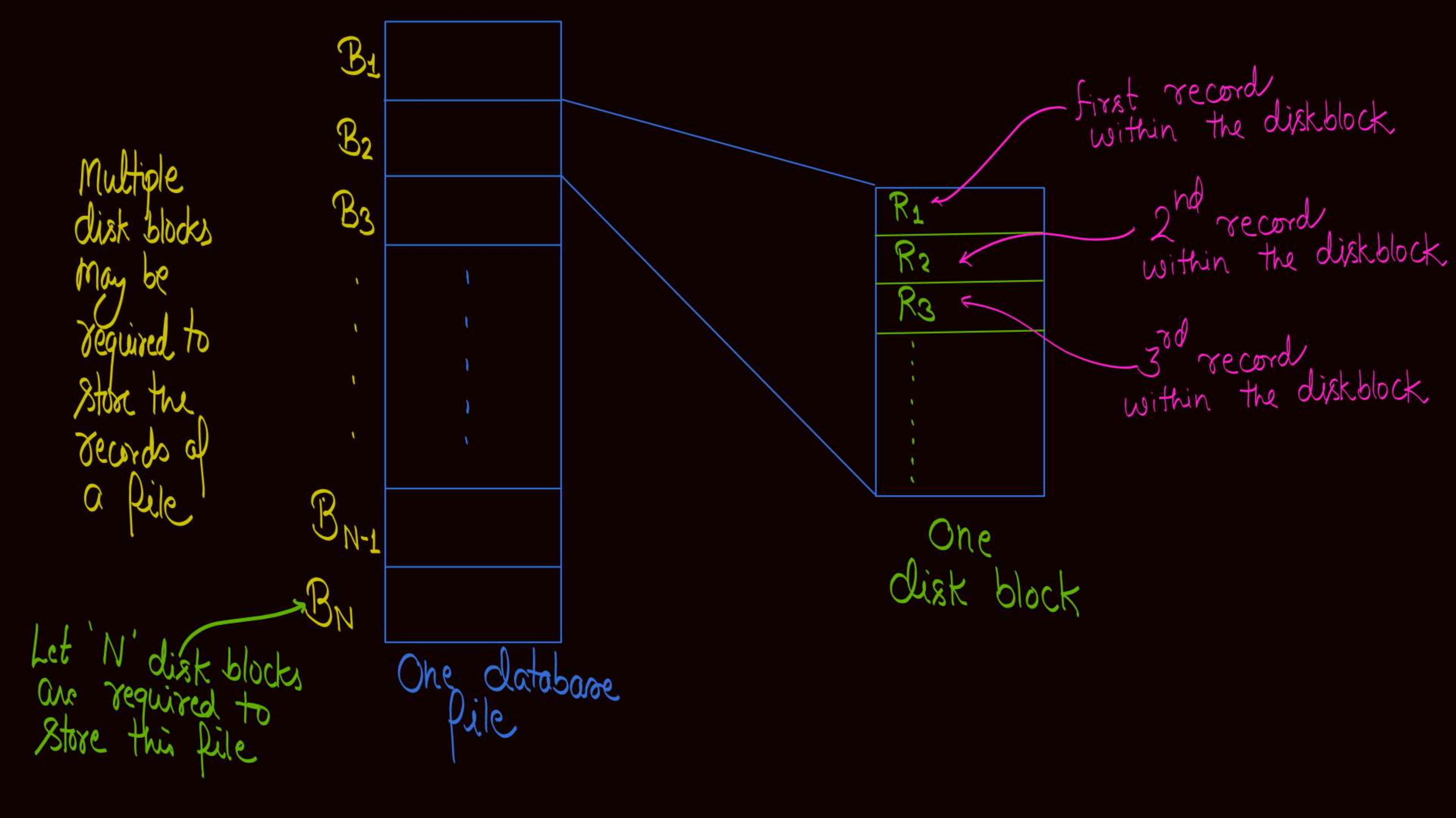
One disk block can be used to store the records of a single file.

↳ Records of different files can not be stored in the same disk block



- Each disk block within the disk will be of same size
- Each block of the disk is identified by a unique number known as "disk block address"
- Each disk block can store multiple records of the same file (not from different files)




Multiple disk blocks may be required to store the records of a file
B_1
B_2
B_3
B_{N-1}
B_N
Let 'N' disk blocks are required to store this file
One database file
R_1
R_2
R_3
First record within the disk block
2nd record within the disk block
3rd record within the disk block
One disk block



Topic : Types of Records

* Two types of records are possible in file system

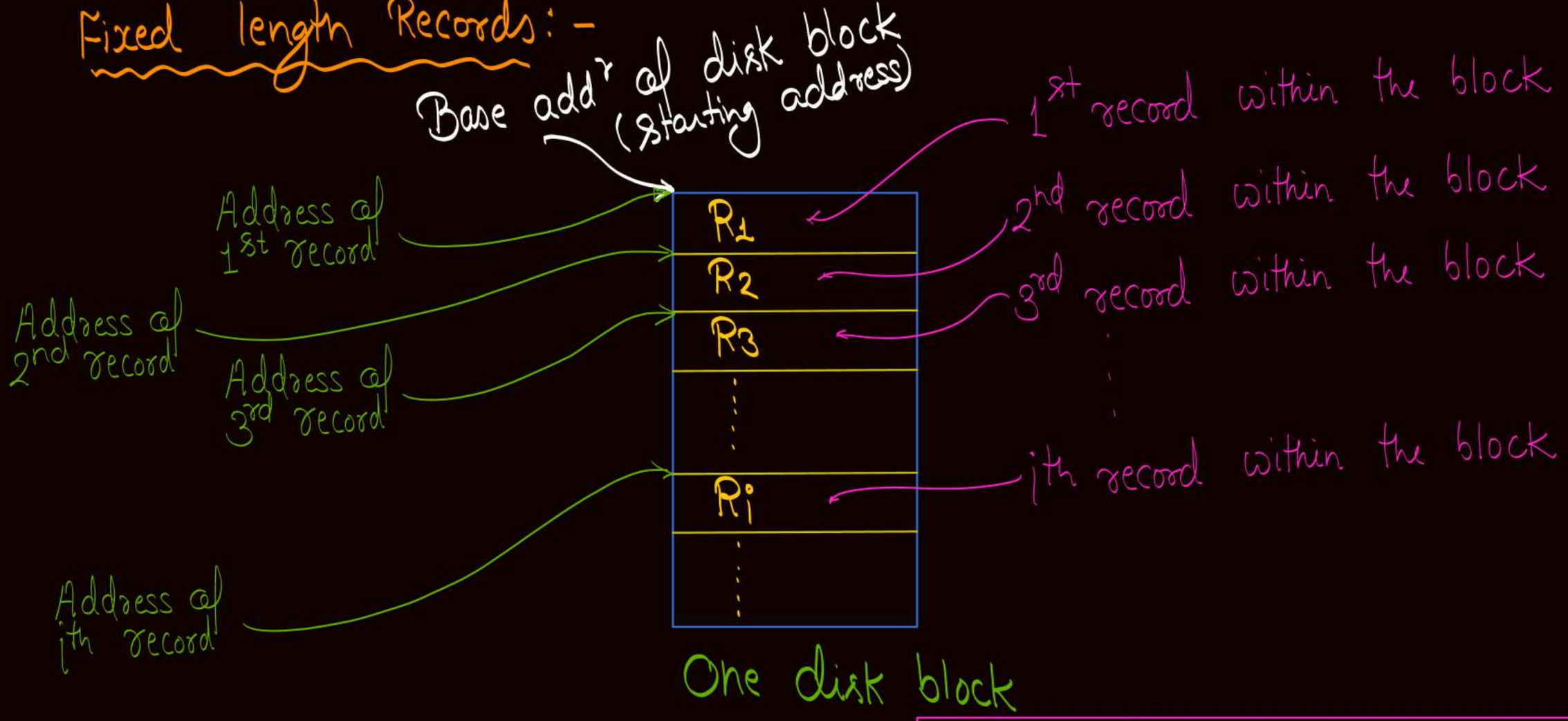
1. Fixed length records
2. Variable length records.



Topic : Fixed length records

When each record of the file is of same size

Fixed length Records :-



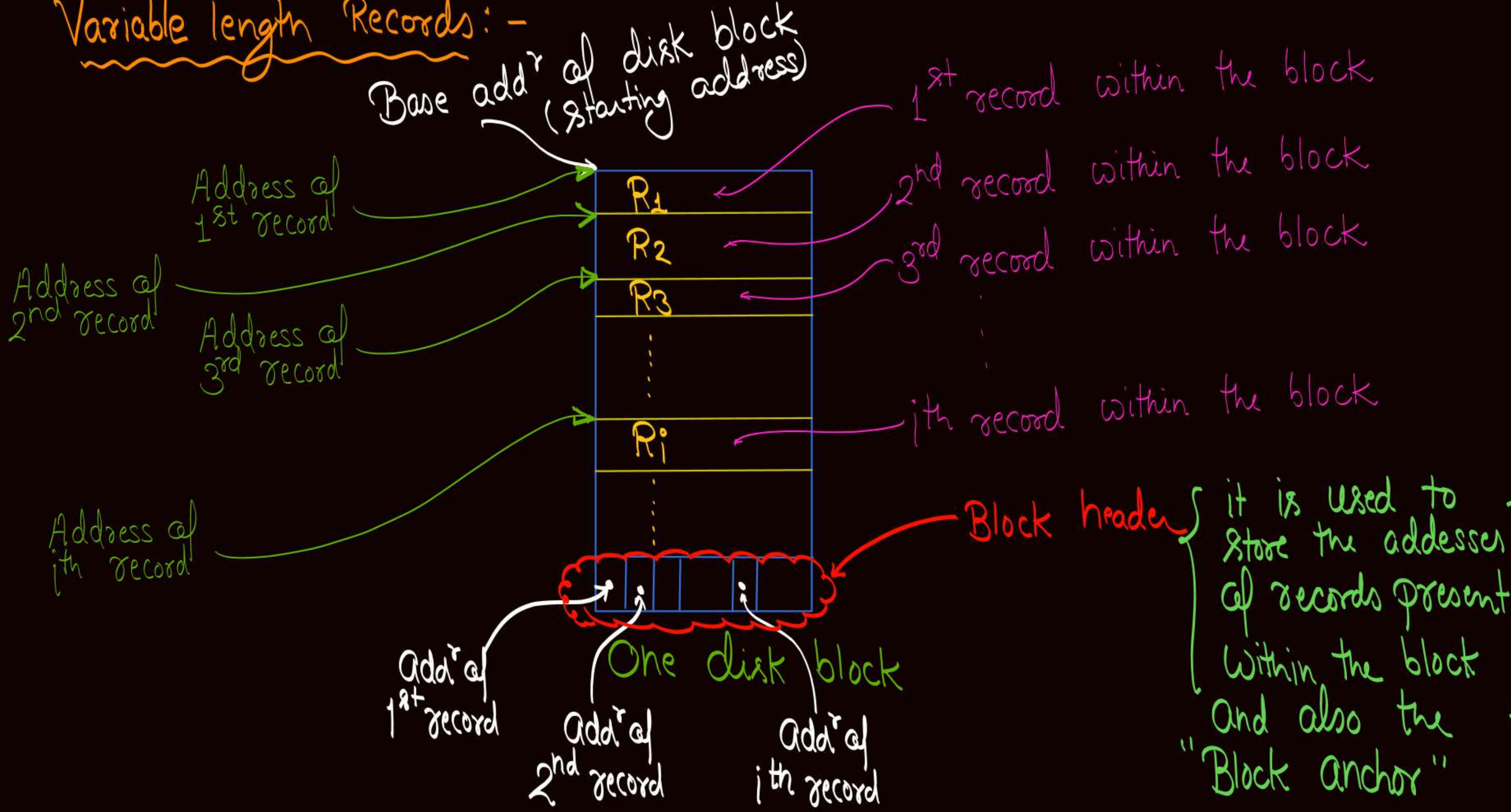
$$\text{Address of } i^{th} \text{ record within the block} = \text{Base add}^{r} \text{ of the block} + (i-1) * \text{Size of each record}$$



Topic : Variable length records

When records of the same file may be of different sizes.

Variable length Records :-



"Block anchor" :- Block anchor gives the address of the next block of the disk used to store the records of the file.

* Note:- Block header may be required with fixed length records as well, in order to store the address of the next disk block of the file {i.e; to store block anchor}

Note:→ ① In the questions if nothing is given about block header size and/or block anchor size then consider it as "Zero", i.e., the complete disk block size with be used to store the records of the file.

② In the question, If block header size is given, then Effective space available in the disk block to store the records of the file will be

= (Disk block size − Block header size)



2 mins Summary

- **Topic** Time stamp ordering protocol with Thomas write rule
- **Topic** Wait-die protocol for deadlock prevention
- **Topic** Wound-wait protocol for deadlock prevention
- **Topic** Database, File and Records
- **Topic** Organization of Records

THANK - YOU